studeersnel

Tìm hiểu về giải thuật mã hóa công khai RSA

Quản Lý An Toàn Thông Tin (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studeersnel

Studocu is not sponsored or endorsed by any college or university
Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



Môn: CƠ SỞ AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

Tìm hiểu về giải thuật mã hóa công khai RSA

Thành viên	Nhóm 05 – Lớp 02
Nguyễn Xuân Bảo Nam	- B22DCAT205
Phạm Thị Lệ Quyên	- B22DCAT241
Đỗ Đức Thái	- B22DCAT277
Đỗ Chí Tùng	- B22DCAT273
Vũ Quang Vinh	- B22DCAT315
Giảng viên:	Đinh Trường Duy

Hà Nội, 10/2024

Dit document is beschikbaar op studeersnel
Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

MỤC LỤC

LỜI MỞ ĐẦU 3
I. Giới thiệu chung 4
II. Lịch sử 4
III. Mô tả sơ lược 4
IV. Thuật toán 5
1. Sinh khóa 5
2. Mã hóa 7
3. Giải mã 7
4. Ví dụ 7
5. Chuyển đổi văn bản rõ 8
V. Điểm yếu của mã hóa RSA 9
VI. Các dạng tấn công mã hóa RSA 11
VII. Phòng chống tấn công mã hóa RSA 12
Tài liệu tham khảo 14

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

LỜI MỞ ĐẦU

Trong bối cảnh xã hội thông tin hiện nay, việc đảm bảo an toàn và bảo mật thông tin đang trở thành một vấn đề cấp thiết. Các giao dịch trực tuyến, trao đổi dữ liệu quan trọng, hay đơn giản là bảo vệ thông tin cá nhân đều đòi hỏi những phương thức mã hóa hiệu quả và đáng tin cậy. Trong số đó, giải thuật mã hóa công khai RSA nổi lên như một giải pháp then chốt, được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau.

Tiểu luận này được thực hiện trong khuôn khổ môn học Cơ sở An toàn thông tin, dưới sự hướng dẫn của thầy Đinh Trường Duy. Nội dung tiểu luận sẽ tập trung vào tìm hiểu giải thuật mã hóa công khai RSA, bao gồm giải thuật sinh khóa, mã hóa, giải mã, các điểm yếu, các dạng tấn công vào RSA và phòng chống. Qua đó, giúp người đọc có cái nhìn tổng quan về vai trò quan trọng của RSA trong việc đảm bảo an toàn thông tin trong thời đại số.

Chúng em xin gửi lời cảm ơn chân thành tới thầy Đinh Trường Duy, người đã tận tình truyền đạt kiến thức, hướng dẫn chúng em trong quá trình nghiên cứu và hoàn thiện tiểu luận. Không chỉ dừng lại ở việc hướng dẫn chuyên môn, Thầy còn luôn khuyến khích và tạo điều kiện cho sinh viên phát huy khả năng tư duy độc lập, sáng tạo, tìm tòi những giải pháp mới. Sự quan tâm và động viên của Thầy đã giúp nhóm em tự tin hơn và hoàn thành bài tiểu luận một cách tốt nhất.

Dit document is beschikbaar op studeersnel

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

I. Giới thiệu chung

Trong mật mã học, **RSA** là một thuật toán mật mã hóa khóa công khai. Đây là thuật toán đầu tiên phù hợp với việc tạo ra chữ ký điện tử đồng thời với việc mã hóa. Nó đánh dấu một sự tiến bộ vượt bậc của lĩnh vực mật mã học trong việc sử dụng khóa công cộng. RSA đang được sử dụng phổ biến trong thương mại điện tử và được cho là đảm bảo an toàn với điều kiện độ dài khóa đủ lớn.

II. Lịch sử

- Thuật toán được Ron Rivest, Adi Shamir và Len Adleman mô tả lần đầu tiên vào năm 1977 tại Học viện Công nghệ Massachusetts (MIT). Tên của thuật toán lấy từ 3 chữ cái đầu của tên 3 tác giả.

- Trước đó, vào năm 1973, Clifford Cocks, một nhà toán học người Anh làm việc tại GCHQ, đã mô tả một thuật toán tương tự. Với khả năng tính toán tại thời điểm đó thì thuật toán này không khả thi và chưa bao giờ được thực nghiệm. Tuy nhiên, phát minh này chỉ được công bố vào năm 1997 vì được xếp vào loại tuyệt mật.

- Thuật toán RSA được MIT đăng ký bằng sáng chế tại Hoa Kỳ vào năm 1983 (Số đăng ký 4.405.829). Bằng sáng chế này hết hạn vào ngày 21 tháng 9 năm 2000. Tuy nhiên, do thuật toán đã được công bố trước khi có đăng ký bảo hộ nên sự bảo hộ hầu như không có giá trị bên ngoài Hoa Kỳ. Ngoài ra, nếu như công trình của Clifford Cocks đã được công bố trước đó thì bằng sáng chế RSA đã không thể được đăng ký.

III. Mô tả sơ lược

- Thuật toán RSA có hai khóa: khóa công khai - public key (hay khóa công cộng) và khóa bí mật - secret key (hay khóa cá nhân). Mỗi khóa là những số cố định sử dụng trong quá trình mã hóa và giải mã. Khóa công khai được công bố rộng rãi cho mọi người và được dùng để mã hóa. Những thông tin được mã hóa bằng khóa công khai chỉ có thể được giải mã bằng khóa bí mật tương ứng. Nói cách khác, mọi người đều có thể mã hóa nhưng chỉ có người biết khóa cá nhân (bí mật) mới có thể giải mã được.

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)



H. Minh họa

- Ta có thể mô phỏng trực quan một hệ mật mã khoá công khai như sau: Bình muốn gửi cho An một thông tin mật mà Bình muốn duy nhất An có thể đọc được. Để làm được điều này, An gửi cho Bình một chiếc hộp có khóa đã mở sẵn và giữ lại chìa khóa. Bình nhận chiếc hộp, cho vào đó một tờ giấy viết thư bình thường và khóa lại (như loại khoá thông thường chỉ cần sập chốt lại, sau khi sập chốt khóa ngay cả Bình cũng không thể mở lại được-không đọc lại hay sửa thông tin trong thư được nữa). Sau đó Bình gửi chiếc hộp lại cho An. An mở hộp với chìa khóa của mình và đọc thông tin trong thư. Trong ví dụ này, chiếc hộp với khóa mở đóng vai trò khóa công khai, chiếc chìa khóa chính là khóa bí mật.

IV. Thuật toán

1. Sinh khóa

Quá trình sinh khóa RSA được thiết kế sao cho việc tìm ra khóa bí mật từ khóa công khai là một bài toán khó giải quyết trong thời gian hợp lý, trừ khi người ta có được các thông tin bí mật liên quan (như các thừa số nguyên tố của modulus n).

Quá trình sinh khóa trong RSA bao gồm các bước sau:

a. Chọn hai số nguyên tố lớn p và q

- RSA bắt đầu bằng việc chọn hai số nguyên tố ngẫu nhiên và lớn, ký hiệu là p và q. Hai số này phải đủ lớn để đảm bảo tính bảo mật.
- Kích thước của p và q thường là hàng trăm, thậm chí hàng nghìn bit. Ví dụ, với một hệ thống RSA 2048 bit, mỗi số nguyên tố p và q có kích thước khoảng 1024 bit.

b. Tính n, là tích của p và q

- Tính toán n như sau: $n = p \times q$

Dit document is beschikbaar op studeersnel
Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

- Giá trị n được gọi là modulus và sẽ được sử dụng cho cả quá trình mã hóa và giải mã.

c. Tính hàm số phi Euler $\phi(n)$

- Hàm phi Euler của n, ký hiệu là $\phi(n)$ được tính như sau: $\phi(n) = (p-1)\times(q-1)$
- Hàm phi Euler đại diện cho số lượng các số nguyên dương nhỏ hơn n mà không có ước số chung với n.

d. Chọn một số nguyên e

- Chọn một số nguyên e sao cho: $1< e < \phi(n)$

  và $\gcd(e,\phi(n)) = 1$ (tức là e và $\phi(n)$ nguyên tố cùng nhau).
- Số e này sẽ là khóa công khai dùng để mã hóa. Một giá trị phổ biến của e là 65537, vì nó đủ lớn để bảo mật nhưng vẫn đảm bảo hiệu quả tính toán.

e. Tính khóa bí mật d

- Khóa bí mật d được tính dựa trên e và $\phi(n)$ sao cho: $d\times e \equiv 1 \pmod{\phi(n)}$
- Điều này có nghĩa là d là nghịch đảo modular của e theo $\phi(n)$. Để tính d, người ta thường sử dụng thuật toán Euclid mở rộng (Extended Euclidean Algorithm).

f. Khóa công khai và khóa bí mật

- Khóa công khai: Là cặp (e,n). Khóa này được công khai và dùng để mã hóa các thông điệp.
- Khóa bí mật: Là cặp (d,n). Khóa này phải được giữ bí mật và dùng để giải mã các thông điệp đã được mã hóa bằng khóa công khai tương ứng.

Ví Dụ Minh Họa

Dưới đây là một ví dụ đơn giản về quá trình sinh khóa RSA:

a. Chọn hai số nguyên tố p và q

- Giả sử chọn $p = 61$ và $q = 53$.

b. Tính n

- $n = 61 \times 53 = 3233$.

c. Tính $\phi(n)$

- $\phi(n) = (61-1) \times (53-1) = 60 \times 52 = 3120$.

d. Chọn e

- Giả sử chọn $e = 17$, vì $\gcd(17, 3120) = 1$.

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

e. Tính khóa bí mật d

- Sử dụng thuật toán Euclid mở rộng, tìm d sao cho: d × 17 ≡ 1 (mod 3120).
- Ta tính được d = 2753.

f. Kết quả:

- Khóa công khai: (e = 17, n = 3233).
- Khóa bí mật: (d = 2753, n = 3233).

2. Mã hóa

Quá trình mã hóa một thông điệp trong RSA sử dụng khóa công khai (e, n). Giả sử M là thông điệp cần mã hóa (được chuyển đổi thành một số nguyên M sao cho M < n), quá trình mã hóa sẽ được thực hiện như sau: $C = M^e \ (mod\ n)$

Trong đó:

- M là thông điệp gốc (plaintext).
- C là bản mã (ciphertext).

3. Giải mã

- Sau khi nhận được bản mã C, người nhận sử dụng khóa riêng để giải mã và khôi phục thông điệp gốc.
- Thông điệp gốc M được tính bằng cách: $M = C^d \ (mod\ n)$
- Nhờ vào cách mà d được tính, phương trình này sẽ khôi phục chính xác thông điệp ban đầu.

4. Ví dụ

Để kết hợp với phần sinh khóa và kết hợp mã hóa, giải mã có một ví dụ dưới đây

Giả sử bạn có cặp khóa RSA với các tham số:

- p=61, q=53
- n = p × q = 61 × 53 = 3233
- ϕ(n) = (61−1) × (53−1) = 3120
- Chọn e = 17 (thoả mãn gcd(17, 3120) = 1).
- Tính d = 2753, là nghịch đảo modular của e theo modulo 3120.

Khóa công khai là (n, e) = (3233, 17), và khóa riêng là (n, d) = (3233, 2753).

- **Mã hóa**: Giả sử thông điệp là M = 65. Bản mã C sẽ được tính như sau:

$$C = 65^{17}\ mod\ 3233 = 2790$$

Dit document is beschikbaar op studeersnel
Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

- **Giải mã**: Khi nhận được C = 2790, sử dụng khóa riêng để giải mã:

$$M = 2790^{2753}\ mod\ 3233 = 65$$

Thông điệp gốc M = 65 đã được khôi phục chính xác.

5. Chuyển đổi văn bản rõ

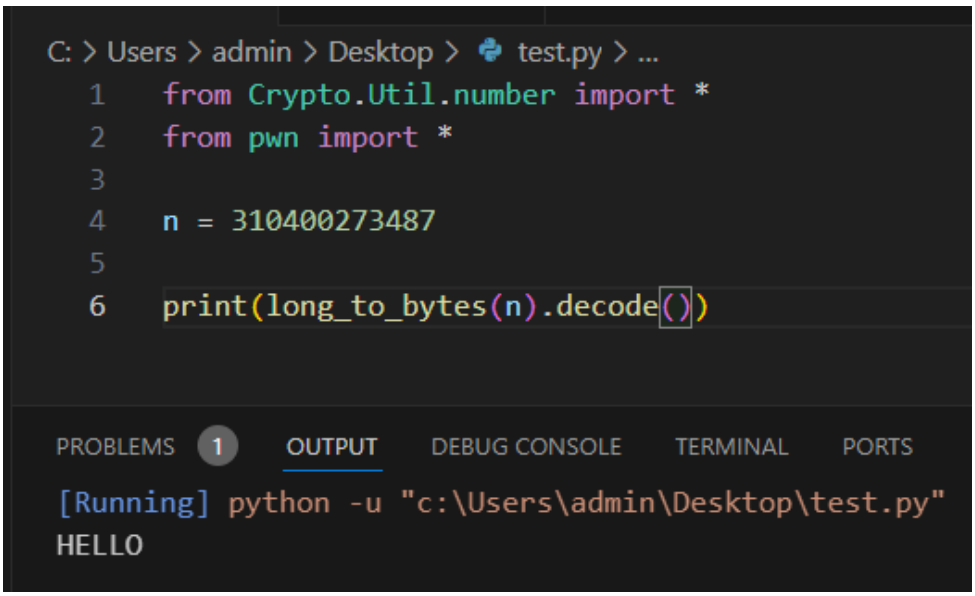
Các hệ thống mật mã như RSA hoạt động trên các con số, nhưng các thông điệp được tạo thành từ các ký tự. Chúng ta nên chuyển đổi các thông điệp của mình thành các con số như thế nào để có thể áp dụng các phép toán?

Cách phổ biến nhất là lấy các byte thứ tự của thông điệp, chuyển đổi chúng thành hệ thập lục phân và nối lại. Điều này có thể được hiểu là một số cơ số 16/hệ thập lục phân và cũng được biểu diễn ở hệ cơ số 10/hệ thập phân.

```
message: HELLO
ascii bytes: [72, 69, 76, 76, 79]
hex bytes: [0x48, 0x45, 0x4c, 0x4c, 0x4f]
base-16: 0x48454c4c4f
base-10: 310400273487
```

Ví dụ:

- Với đoạn massage HELLO được tạo như bên trên. Mình sẽ tiến hành giải mã ngược lại theo các bước. Hoặc mình có thể sử dung hàm `long_to_bytes()` trong thư viện pwntools của Python để có thể xử lý một cách dễ dàng hơn



Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

V. Điểm yếu của mã hóa RSA

RSA là một trong những thuật toán mã hóa khóa công khai phổ biến nhất, nhưng nó vẫn có một số điểm yếu tiềm ẩn mà nếu không được chú ý, có thể bị khai thác. Dưới đây là các điểm yếu chính của RSA:

1. Kích thước khóa không đủ lớn:

- Kích thước khóa nhỏ: Nếu các số nguyên tố được chọn quá nhỏ, kẻ tấn công có thể sử dụng các phương pháp như **phân tích thừa số nguyên tố** (prime factorization) để tìm ra các khóa riêng tư một cách nhanh chóng. Ví dụ, khóa RSA 512-bit hiện nay có thể bị phá vỡ khá dễ dàng với công nghệ hiện đại. Do đó, kích thước khóa khuyến nghị hiện tại thường là 2048-bit hoặc cao hơn để đảm bảo an toàn.

2. Cùng số mũ công khai và khóa công khai trên nhiều hệ thống:

- RSA Common Modulus Attack xảy ra khi cùng một số mũ công khai được sử dụng trên nhiều thông điệp hoặc trên nhiều hệ thống khác nhau, điều này có thể dẫn đến việc giải mã được thông điệp mà không cần biết khóa riêng tư.
- Low Encryption Exponent (như khi sử dụng e=3): Nếu giá trị mũ công khai nhỏ (như 3) và nếu thông điệp không được padding đúng cách hoặc ngắn, có thể dẫn đến tấn công giải mã trực tiếp bằng cách sử dụng căn bậc e của thông điệp đã mã hóa.

3. Padding không an toàn:

- Nếu không sử dụng padding thích hợp (ví dụ như **PKCS#1 v1.5 padding**), có thể dẫn đến các cuộc tấn công kiểu **Bleichenbacher attack** (tấn công oracle) cho phép kẻ tấn công giải mã các thông điệp được mã hóa bằng RSA mà không cần biết khóa riêng tư.

4. Tấn công dựa trên việc sử dụng khóa yếu:

- Nếu các số nguyên tố được chọn để sinh khóa không ngẫu nhiên hoặc có quan hệ gần gũi (ví dụ: số nguyên tố gần nhau), điều này có thể dẫn đến việc phá vỡ RSA thông qua việc phân tích thừa số nhanh hơn.

5. Tấn công dựa trên thời gian (Timing Attack):

- Timing attacks khai thác thời gian thực hiện các phép toán trong quá trình giải mã. Nếu kẻ tấn công có thể đo đạc thời gian giải mã một cách chính xác, họ có thể dần dần xác định được khóa riêng tư.

6. Tấn công bằng lỗi phần cứng:

Dit document is beschikbaar op studeersnel

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

- RSA cũng có thể bị tấn công khi phần cứng bị lỗi. Một cuộc tấn công như **Fault Injection Attack** có thể lợi dụng lỗi phần cứng để làm giảm tính bảo mật của quá trình mã hóa hoặc giải mã RSA.

7. Tấn công dựa trên thông tin về quá trình mã hóa/gửi đi (Side-channel attacks):

- Các cuộc tấn công này không nhắm trực tiếp vào thuật toán mà nhắm vào các kênh rò rỉ thông tin, ví dụ như năng lượng tiêu thụ, bức xạ điện từ, hoặc thậm chí là âm thanh từ phần cứng khi thực hiện quá trình mã hóa hoặc giải mã.

8. Tấn công dựa trên thông điệp lặp lại (Chosen Ciphertext Attack):

- Với các biến thể của RSA không sử dụng padding an toàn, kẻ tấn công có thể yêu cầu giải mã một thông điệp được mã hóa nhiều lần và từ đó suy luận ra khóa riêng tư hoặc nội dung của thông điệp.

9. Khóa công khai không được xác minh đúng cách:

- Nếu không có biện pháp kiểm tra tính hợp lệ của khóa công khai (ví dụ như chữ ký số hoặc hệ thống chứng chỉ số), kẻ tấn công có thể gửi một khóa công khai giả mạo và lừa người dùng mã hóa thông điệp bằng khóa đó. Điều này có thể dẫn đến việc kẻ tấn công có thể giải mã được thông điệp.

10. Lỗi trong việc chọn số nguyên tố:

- RSA yêu cầu sử dụng hai số nguyên tố lớn để tạo khóa, nhưng nếu việc chọn số nguyên tố này không đủ ngẫu nhiên hoặc có tính chất dễ đoán (ví dụ như khoảng cách giữa hai số nguyên tố nhỏ), điều này có thể dẫn đến việc phá vỡ khóa RSA.

Cách khắc phục:

- Sử dụng kích thước khóa đủ lớn (ít nhất 2048-bit hoặc 3072-bit).
- Sử dụng các thuật toán padding an toàn như **OAEP (Optimal Asymmetric Encryption Padding)**.
- Tránh sử dụng các giá trị mũ công khai nhỏ, đặc biệt là e = 3.
- Sử dụng các thư viện mã hóa đã được kiểm chứng và đảm bảo kiểm tra lỗ hổng bảo mật thường xuyên.
- Sử dụng các kỹ thuật bảo mật phần cứng và bảo vệ trước các cuộc tấn công dựa trên thời gian.

=> RSA tuy mạnh nhưng cần được triển khai đúng cách để tránh các điểm yếu tiềm ẩn này.

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

VI. Các dạng tấn công mã hóa RSA

RSA (Rivest-Shamir-Adleman) là một thuật toán mã hóa khóa công khai, nhưng nó không phải là bất khả xâm phạm. Dưới đây là một số dạng tấn công phổ biến vào RSA:

1. Tấn công dựa trên số nguyên tố yếu (Factorization Attack)

RSA phụ thuộc vào việc nhân hai số nguyên tố lớn với nhau. Nếu hai số này nhỏ hoặc không đủ mạnh, kẻ tấn công có thể sử dụng các thuật toán phân tích số nguyên tố như **General Number Field Sieve (GNFS)** để phân tích ra các thừa số ban đầu, từ đó phá vỡ RSA.

- Tấn công bằng Fermat: Nếu hai số nguyên tố tạo thành modulus n = p * q có giá trị gần nhau, thì thời gian phân tích sẽ giảm đi đáng kể.

2. Tấn công thời gian (Timing Attack)

Tấn công này dựa trên thời gian thực hiện phép toán mã hóa hoặc giải mã. Một kẻ tấn công có thể đo thời gian thực hiện các phép tính và từ đó suy ra khóa bí mật. Nếu thời gian thực hiện không được bảo vệ đúng cách, điều này có thể làm lộ thông tin về khóa.

3. Tấn công văn bản mã hóa chọn lọc (Chosen Ciphertext Attack - CCA)

Trong tấn công này, kẻ tấn công có khả năng chọn các văn bản mã hóa và yêu cầu nạn nhân giải mã chúng. Với thông tin từ các văn bản giải mã, kẻ tấn công có thể dần dần suy ra khóa giải mã.

- Bleichenbacher's Attack: Là một ví dụ nổi tiếng về CCA nhắm vào RSA trong chế độ mã hóa Padding Oracle.

4. Tấn công dựa trên khóa công khai yếu (Low Public Exponent Attack)

Trong RSA, khóa công khai e thường được chọn là một số nhỏ (thường là 3 hoặc 65537) để giảm thiểu thời gian mã hóa. Nếu không có biện pháp bảo vệ phù hợp, việc chọn e = 3 có thể khiến RSA dễ bị tấn công.

- Tấn công với các bản mã nhỏ (Small Exponent Attack): Nếu thông điệp (plaintext) nhỏ hơn modulus n, và sử dụng khóa công khai nhỏ, việc tính toán $m^e \ mod \ n$ có thể không thực sự tạo ra một giá trị lớn hơn n, điều này cho phép kẻ tấn công khôi phục bản rõ dễ dàng.

5. Tấn công lặp lại văn bản (Replay Attack)

Nếu cùng một thông điệp được mã hóa nhiều lần với cùng một khóa công khai, kẻ tấn công có thể phân tích sự khác biệt giữa các bản mã để suy ra thông tin về bản rõ.

Dit document is beschikbaar op studeersnel

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

6. Tấn công dựa trên số dư Trung Hoa (Chinese Remainder Theorem Attack - CRT Attack)

Nếu kẻ tấn công có thể tìm được các bản mã được mã hóa bằng cùng một thông điệp với các modulus khác nhau, có thể sử dụng **Định lý phần dư Trung Hoa** để khôi phục bản rõ.

7. Tấn công bằng lỗi phần cứng (Fault Attack)

Đây là một dạng tấn công vật lý, nơi kẻ tấn công cố ý tạo ra lỗi trong quá trình tính toán RSA bằng cách thay đổi các tham số phần cứng hoặc môi trường, sau đó phân tích kết quả của lỗi để khôi phục thông tin về khóa bí mật.

8. Tấn công dựa trên thông tin rò rỉ (Side-channel Attack)

Những tấn công này không nhằm vào bản thân thuật toán RSA, mà nhằm vào việc phân tích các thông tin như tiêu thụ năng lượng, bức xạ điện từ, hay âm thanh phát ra từ máy tính trong quá trình tính toán.

9. Tấn công bằng phương pháp giải mã modulo yếu (Low-Entropy Moduli Attack)

Nếu modulus n có các đặc tính không ngẫu nhiên hoặc có mối liên hệ với các modulus khác (ví dụ, chung một số thừa số với modulus khác), kẻ tấn công có thể phân tích chúng và tấn công vào hệ thống.

VII. Phòng chống tấn công mã hóa RSA

Để bảo vệ RSA khỏi các dạng tấn công đã nêu, cần áp dụng nhiều biện pháp bảo mật và tối ưu hóa quá trình triển khai. Dưới đây là các cách phòng chống cho từng dạng tấn công:

1. Phòng chống tấn công dựa trên số nguyên tố yếu (Factorization Attack)

- Chọn số nguyên tố lớn: Sử dụng số nguyên tố lớn cho p và q, thông thường modulus n phải có kích thước ít nhất là 2048 bit để đảm bảo an toàn.
- Tránh các số nguyên tố gần nhau: Đảm bảo hai số nguyên tố không quá gần nhau để tránh tấn công bằng phương pháp Fermat.

2. Phòng chống tấn công thời gian (Timing Attack)

- Thực hiện phép tính với thời gian không đổi: Tối ưu hóa thuật toán để mọi phép tính RSA (mã hóa, giải mã) mất cùng thời gian, bất kể giá trị của dữ liệu vào.

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

- Thêm độ trễ ngẫu nhiên: Bổ sung các thao tác ngẫu nhiên hoặc giả ngẫu nhiên để làm chậm quá trình tính toán, khiến cho kẻ tấn công khó có thể đoán được thời gian chính xác.

3. Phòng chống tấn công văn bản mã hóa chọn lọc

(Chosen Ciphertext Attack - CCA)

- Sử dụng padding an toàn: Sử dụng các phương pháp padding an toàn như **Optimal Asymmetric Encryption Padding (OAEP)**. OAEP là một phương pháp padding đã được chứng minh bảo vệ chống lại CCA.
- Sử dụng chữ ký số: Khi sử dụng RSA để mã hóa, cần kết hợp với chữ ký số hoặc các phương pháp xác thực khác để đảm bảo không ai có thể tạo văn bản mã hóa giả mạo.

4. Phòng chống tấn công dựa trên khóa công khai yếu (Low Public Exponent Attack)

- Sử dụng e = 65537: Đây là một giá trị phổ biến và an toàn cho e, nó đảm bảo sự cân bằng giữa tốc độ mã hóa và an toàn.
- Sử dụng padding OAEP: Đảm bảo luôn sử dụng padding khi mã hóa để phòng tránh các vấn đề liên quan đến bản rõ nhỏ.

5. Phòng chống tấn công lặp lại văn bản (Replay Attack)

- Sử dụng các kỹ thuật chống replay: Mã hóa mỗi thông điệp với một giá trị nonce hoặc một giá trị salt ngẫu nhiên để mỗi bản mã là duy nhất, ngay cả khi mã hóa cùng một thông điệp nhiều lần.

6. Phòng chống tấn công dựa trên số dư Trung Hoa (CRT Attack)

- Tránh mã hóa cùng thông điệp với các modulus khác nhau: Không sử dụng cùng một thông điệp với nhiều khóa công khai khác nhau mà không có biện pháp phòng ngừa.
- Sử dụng các chuẩn chữ ký và mã hóa an toàn: Hạn chế sử dụng các phương pháp mã hóa không được chuẩn hóa, đặc biệt là trong môi trường đa người dùng hoặc mạng lưới.

7. Phòng chống tấn công bằng lỗi phần cứng (Fault Attack)

- Sử dụng phần cứng có khả năng chống lỗi: Phần cứng bảo mật nên được thiết kế để phát hiện và tự động khắc phục các lỗi xuất hiện trong quá trình tính toán.

Dit document is beschikbaar op studeersnel
Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)

- Thực hiện kiểm tra sau khi tính toán: Sau khi giải mã hoặc mã hóa, cần kiểm tra lại kết quả bằng cách thực hiện lại quá trình với cùng dữ liệu để đảm bảo không có lỗi phần cứng.

8. Phòng chống tấn công dựa trên thông tin rò rỉ (Side-channel Attack)

- Cách ly thiết bị khỏi môi trường bên ngoài: Hạn chế phát tán các tín hiệu có thể bị khai thác (năng lượng, âm thanh, bức xạ điện từ) thông qua việc cách ly vật lý hoặc bảo mật phần cứng.
- Thực hiện các thao tác giả ngẫu nhiên: Thêm các hoạt động giả ngẫu nhiên vào quá trình tính toán để gây nhiễu và khiến các thông tin bị rò rỉ trở nên vô ích.

9. Phòng chống tấn công bằng phương pháp giải mã modulo yếu (Low-Entropy Moduli Attack)

- Sử dụng các số nguyên tố ngẫu nhiên và không liên quan: Đảm bảo rằng các modulus RSA được tạo ra từ các số nguyên tố không có mối liên hệ với nhau và không được tái sử dụng.
- Thay đổi khóa định kỳ: Đối với các hệ thống sử dụng RSA trong thời gian dài, nên thay đổi khóa định kỳ để giảm thiểu rủi ro của việc modulus bị khai thác.

Các biện pháp bổ sung:

- **Tăng kích thước khóa**: Sử dụng các cặp khóa có kích thước ít nhất 2048 bit, và cân nhắc 3072 hoặc 4096 bit cho các ứng dụng yêu cầu mức độ bảo mật cao.
- **Sử dụng RSA kèm các thuật toán bảo mật khác**: RSA có thể được kết hợp với các phương pháp mã hóa khác như **AES** để tạo ra một hệ thống bảo mật kết hợp (hybrid cryptosystem), giúp tăng cường bảo mật và hiệu suất.
- **Kiểm tra và cập nhật thường xuyên**: Thực hiện kiểm tra bảo mật định kỳ và cập nhật các hệ thống sử dụng RSA để đảm bảo rằng các lỗ hổng mới phát hiện được khắc phục kịp thời.

Những biện pháp trên giúp gia tăng mức độ bảo mật của RSA, hạn chế khả năng bị tấn công từ nhiều phương pháp khác nhau.

Tài liệu tham khảo:

CryptoHack – RSA challenges

RSA (mã hóa) – Wikipedia tiếng Việt

Downloaded by Trà 9c Nguy?n ?ình (nguyendinhtratl9c@gmail.com)